

PKCERT AI & Software Development Internship

Task 08: Classification Models

Abdullah Amir

AI and Software Development
13 July 2026

This report covers the theory and the results. The full code, outputs, and plots live in the notebook `classification_models.ipynb`. The dataset is the **Pima Indians Diabetes** set: 768 patients described by 8 clinical measurements (glucose, BMI, age, and so on), where the goal is to predict whether a patient has diabetes. About 35 percent of the patients are diabetic, so the classes are imbalanced, which is exactly why accuracy on its own will not be enough. A quirk worth flagging up front: several columns use 0 to mean “not recorded”, and since a glucose or BMI of zero is physiologically impossible, we treat those zeros as missing and fill them with the column median before training.

Part A: Logistic Regression

Working principle. Logistic Regression is a linear model for classification. It takes a weighted sum of the features, exactly like linear regression, and then squashes that number through the **sigmoid** function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

which maps any real value into the range 0 to 1 and is read as the probability of the positive class. If that probability is above 0.5 we predict diabetes, otherwise we predict no diabetes. The weights are learned by maximising the likelihood of the observed labels, which is the same as minimising the log-loss, using an iterative solver. Because the model is linear in the features, the sign and size of each weight tell you how each measurement pushes the predicted risk up or down, so the model is easy to interpret.

Real-world application. It is the standard tool for medical risk scoring and credit decisions: estimating a patient’s probability of a disease, whether a transaction is fraudulent, or whether a customer is about to churn. In all of these you want a calibrated probability and a model you can actually explain to a doctor or a regulator, and Logistic Regression gives you both.

Setup. We split the data 80/20 with stratification, so both the training and test sets keep the same 35 percent diabetic ratio, and we standardise the features. Standardising matters here because the columns live on very different scales (insulin runs into the hundreds while the pedigree score sits around 0.5), and the solver converges faster and fairer when everything is on a common scale. The tree models later do not need this step.

Part B: Classification Metrics

Accuracy on its own is misleading with imbalanced classes. A lazy model that always guessed “no diabetes” would already score 65 percent here without learning anything, so we look at the full set of metrics. All four are built from the four cells of the confusion matrix: true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).

- **Accuracy** = $(TP + TN)/\text{all}$. The overall share of correct predictions. A good headline number, but blind to *which* class is being missed.

- **Precision** = $TP/(TP + FP)$. Of the patients we flagged as diabetic, how many really were. High precision means few false alarms.
- **Recall** (sensitivity) = $TP/(TP + FN)$. Of the patients who really had diabetes, how many we caught. This is the metric that matters most in medical screening, because a false negative, telling a sick patient they are healthy, is the costliest error.
- **F1-score** = $2PR/(P + R)$, the harmonic mean of precision and recall. One number that stays low unless *both* are high, so it is the honest summary when you care about both at once.

For the Logistic Regression model the scores on the test set were accuracy **0.708**, precision 0.600, recall 0.500, and F1 0.545, with a ROC-AUC of 0.813. The confusion matrix below shows the split of mistakes: 18 false alarms and 27 missed diabetics. That low recall is the model's weak spot at the default 0.5 threshold, and it is the sort of thing you would tune in a real screening setting.

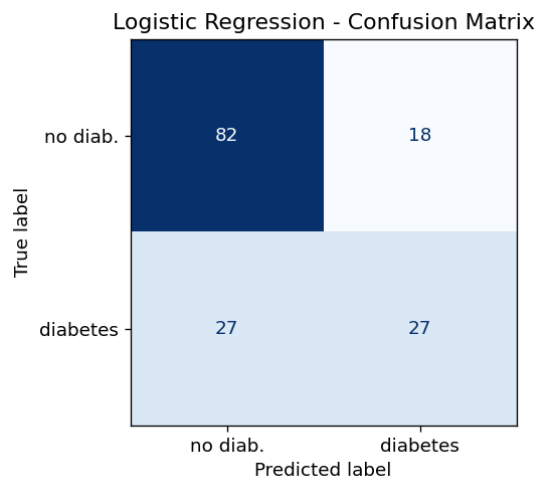


Figure 1: Logistic Regression confusion matrix. Rows are the true class, columns the prediction. The bottom-left cell (27 missed diabetics) is the error we most want to reduce.

Part C: Decision Trees & Random Forests

Decision Tree. A tree asks a sequence of yes/no questions on the features (“is glucose above 128?”) and follows the branches down to a leaf that holds the predicted class. At each split it picks the feature and threshold that best separate the classes, measured by **Gini impurity** or entropy. Trees need no feature scaling, they capture non-linear boundaries, and they are wonderfully easy to read, but a fully grown tree memorises the training data and overfits. We cap the depth at 4 to keep it honest and legible.

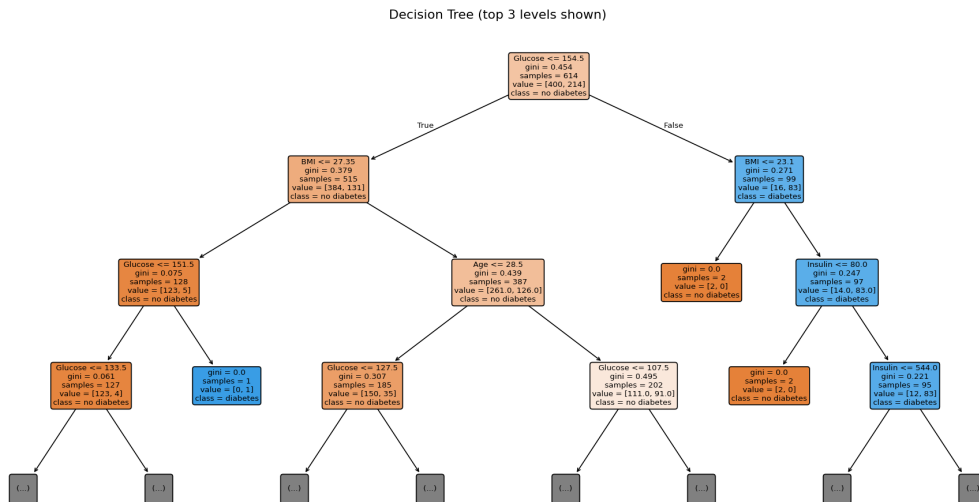


Figure 2: The top three levels of the depth-4 tree. Glucose is the very first split, which matches the clinical intuition.

Random Forest. A forest is an **ensemble** of many decision trees. Each tree is trained on a bootstrap sample of the rows and, at every split, may only look at a random subset of the features. The trees therefore make different mistakes, and averaging their votes cancels out much of the noise. This *bagging* keeps most of a single tree's flexibility while sharply cutting the variance that makes one tree so fragile.

Comparison. On this particular test split the Decision Tree actually came out ahead of both other models, with accuracy 0.786 and F1 0.692, while the Random Forest scored accuracy 0.740 and F1 0.600. Taken at face value that looks like a clear win for the single tree, but the cross-validation in Part D tells a very different and more trustworthy story. The confusion matrices below and the feature importances (glucose first, then BMI, family history, and age) round out the picture.

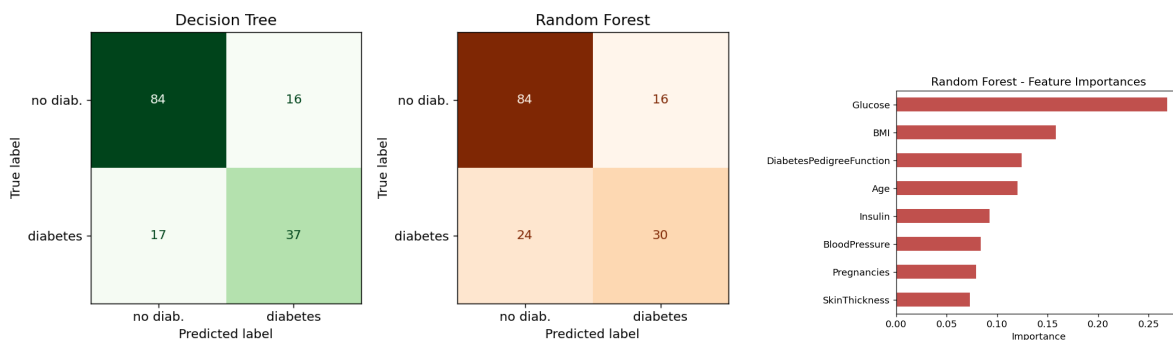


Figure 3: Left: confusion matrices for the tree and the forest. Right: the forest's feature importances, led by glucose and BMI.

Advantages and limitations. A single tree is fast, needs no scaling, and is fully transparent, but it is high-variance and overfits easily. A forest fixes the variance problem and is usually more accurate and far more stable, at the cost of interpretability (it is now 200 trees averaged together) and a bit more compute.

Part D: Comparative Analysis

Putting all three side by side on the same test set, and then backing it up with 5-fold cross-validation and ROC-AUC, gives a fuller read than any single number.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV acc (std)
Logistic Regression	0.708	0.600	0.500	0.545	0.813	0.772 (0.018)
Decision Tree	0.786	0.698	0.685	0.692	0.789	0.732 (0.040)
Random Forest	0.740	0.652	0.556	0.600	0.816	0.761 (0.035)

Table 1: Test-set metrics plus 5-fold cross-validation accuracy. The tree wins the single split but has the lowest, wobbliest cross-validation score.

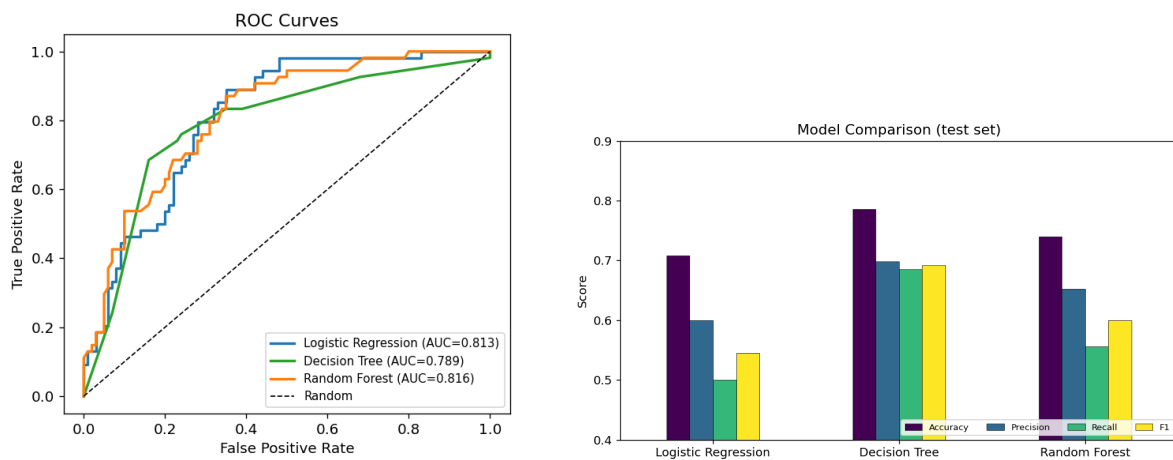


Figure 4: Left: ROC curves, where the forest and Logistic Regression edge ahead. Right: the four headline metrics side by side.

Reading the table. The Decision Tree looks like the winner on this one split, but cross-validation exposes it as the least reliable of the three: its average accuracy across five folds is the lowest at 0.732, and its spread is the widest at 0.040, roughly double the others. That strong test score is mostly luck of the split. Logistic Regression has the best and steadiest cross-validation accuracy (0.772, std 0.018) and a near-top ROC-AUC, but its recall at the default threshold is weak. The Random Forest posts the best ROC-AUC (0.816), the second-best cross-validation accuracy, and far more stability than the lone tree.

Recommendation. *For this dataset I recommend the Random Forest.* ROC-AUC measures how well a model ranks patients by risk regardless of where you put the cut-off, and the forest ranks best. That is what counts in a screening setting, where you would lower the decision threshold to catch more diabetics, trading a little precision for the recall you actually care about. The forest is also far more robust than the single Decision Tree, whose flattering test score does not survive cross-validation. Logistic Regression is a very strong and more interpretable runner-up, and would be my pick if a transparent, explainable risk score mattered more than squeezing out the last bit of ranking accuracy. The lone Decision Tree is best kept for when a fully readable decision path is the priority.

The notebook and README are on GitHub: <https://github.com/AbdullahAmir06/ai-internship-abdullah>